EclipseStack Technical Report

Architecture, Tooling, and Implementation Notes

Simon Knight

Adelaide, Australia

March 2026 • Version 0.1.0

Last updated: March 11, 2026

EclipseStack

Abstract. This technical report documents EclipseStack's architecture, build and release workflow, and implementation decisions. It is intended as a living engineering reference that captures system boundaries, key components, and rationale for design choices.

Contents

1 Overview 2

2 System Architecture 2

3 Build, Test, and Release 2

3.1 Math and Algorithm Notes . . . 2
3.2 Concept Diagrams (Algorithms) . . . 3
3.3 Module-Level Logic (Alignment) . . . 4
3.4 Worked Example (Transform Construction) . . . 4

4 Design Decisions 4

List of Figures 5

List of Tables 5

List of Design Decisions & Rules 6

Revision History 6

1 Overview

This report follows the `sk-techreport` format from `/dev/papers`. Use it to record architectural intent and the reasoning behind changes.

Design Decision 1: Scope

This document focuses on: system architecture, module boundaries, build/test tooling, and design decisions that impact maintainability.

2 System Architecture

Describe the high-level architecture here, including the main subsystems and their interfaces.

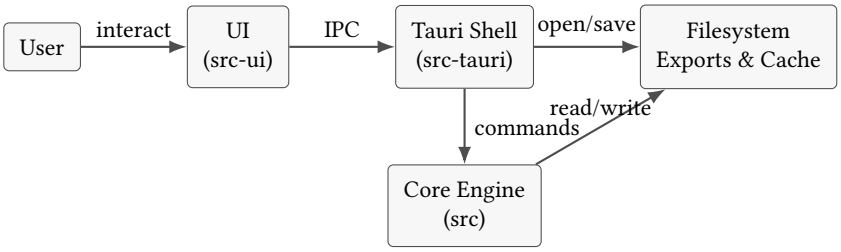

Figure 1. High-level component boundaries and data flow.

Design Decision 2: Repository Layout

Record why directories such as `src`, `src-tauri`, and `src-ui` exist, and how responsibilities are split.

3 Build, Test, and Release

Document build commands, CI expectations, and release steps.

: Reproducible Builds

Builds should be reproducible on developer machines and CI by pinning toolchains and documenting required dependencies.

3.1 Math and Algorithm Notes

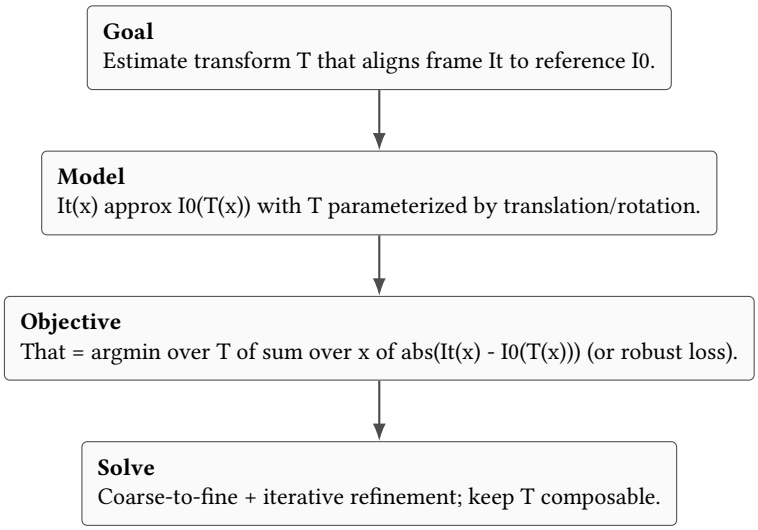

Figure 2. Generic alignment formulation used throughout the pipeline.

3.2 Concept Diagrams (Algorithms)

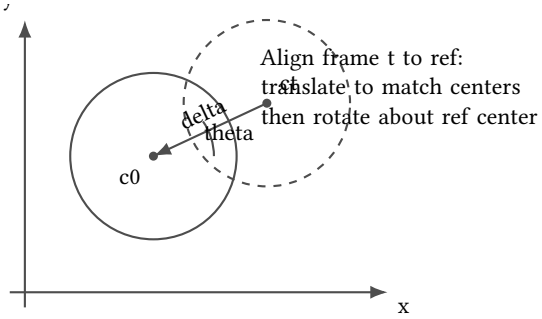

Figure 3. Geometric view of alignment: translation to a shared disk center, then rotation about the reference center.

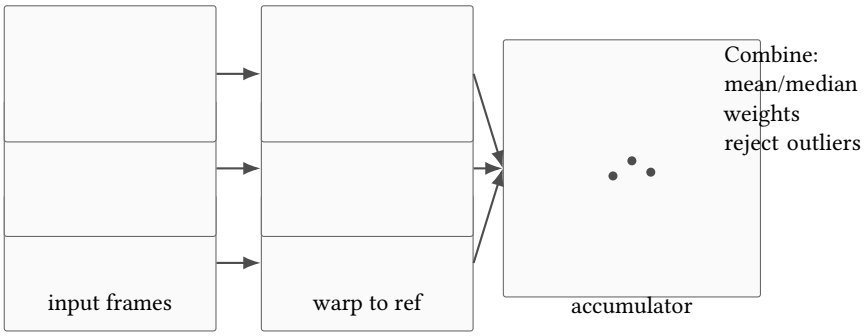

Figure 4. Stacking schematic: warp each frame into the reference coordinate system, then combine in an accumulation buffer.

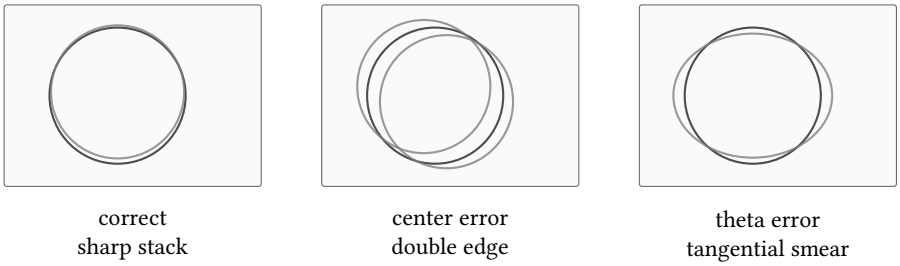

Figure 5. Error modes in stacking: mis-centering causes doubled edges; rotation error causes tangential smear.

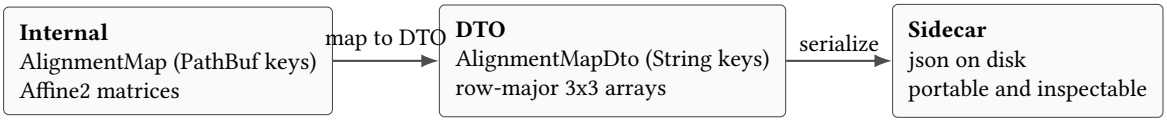

Figure 6. Data contract for IPC and sidecars: deterministic DTO shape and portable persistence.

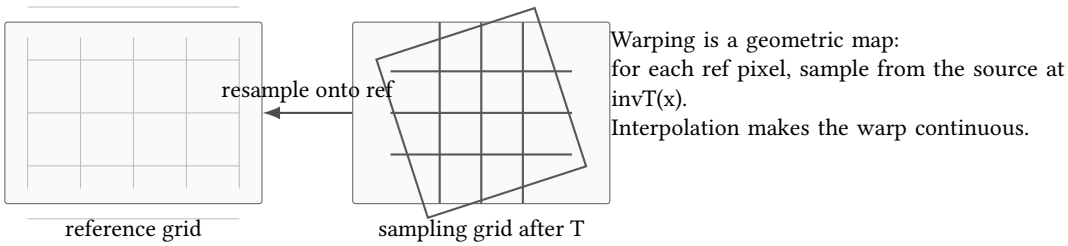

Figure 7. Warping intuition: a transform changes the sampling grid; stacking compares frames only after they share the same reference grid.

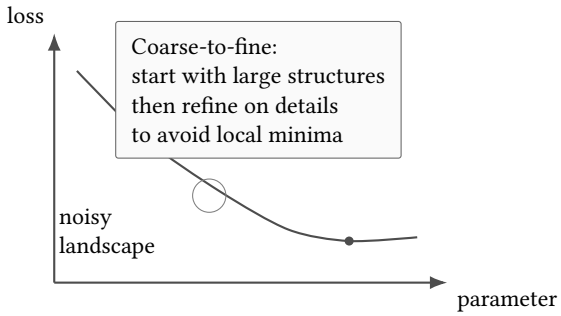

Figure 8. Optimization intuition: coarse-to-fine reduces spurious minima and improves convergence when aligning noisy frames.

3.3 Module-Level Logic (Alignment)

Alignment uses disk detection for robust coarse registration, feature matching for fine translation, and optional rotation estimation when signal quality permits.

3.4 Worked Example (Transform Construction)

Given
center = (center_x, center_y) from `find_disk`; translation = (dx, dy) relative to reference; rotation = angle_deg from `calculate_field_rotation`.

Steps
1) to-origin: translate by (-center_x, -center_y)
2) rotate by angle_deg
3) from-origin: translate by (center_x + dx, center_y + dy)

Compose
affine = from-origin * rotate * to-origin (note: matrix multiplication applies right-to-left).

Figure 9. Worked example: how a per-frame affine transform is constructed.

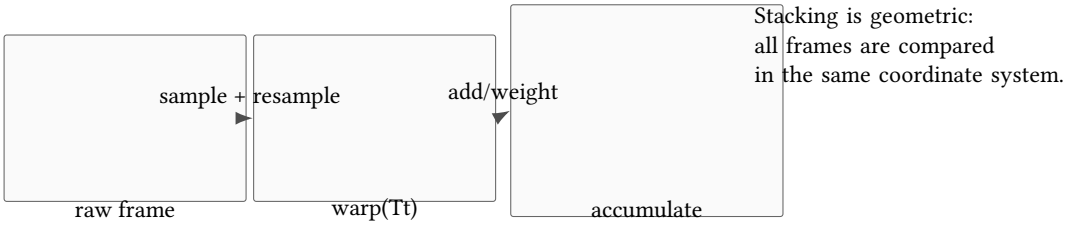

Figure 10. Geometric stacking: warp into the reference coordinate system before accumulation.

4 Design Decisions

Add design decisions as they are made. Keep each one small and focused.

Design Decision 3: Example Decision

Decision: Choose a single source of truth for configuration.
Rationale: Avoid divergence between multiple config formats.
Consequences: Some migrations may be required.

List of Figures

1 High-level component boundaries and data flow. 2
2 Generic alignment formulation used throughout the pipeline.. 2
3 Geometric view of alignment: translation to a shared disk center, then rotation about the reference center.. 3
4 Stacking schematic: warp each frame into the reference coordinate system, then combine in an accumulation buffer. 3
5 Error modes in stacking: mis-centering causes doubled edges; rotation error causes tangential smear. 3
6 Data contract for IPC and sidecars: deterministic DTO shape and portable persistence. 3
7 Warping intuition: a transform changes the sampling grid; stacking compares frames only after they share the same reference grid.. 3
8 Optimization intuition: coarse-to-fine reduces spurious minima and improves convergence when aligning noisy frames. 4
9 Worked example: how a per-frame affine transform is constructed. 4
10 Geometric stacking: warp into the reference coordinate system before accumulation.. 4

List of Tables

List of Design Decisions & Rules

1 Scope 2
2 Repository Layout 2
1 Design Rule: Reproducible Builds 2
3 Example Decision 4

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial scaffold using sk-techreport style